

Rapport de Projet

Five Nights at Fabron

Module R401 — Architecture Logicielle

Durée 3 semaines

Projet Développement d'un jeu vidéo web basé sur une architecture Client-Serveur

Développeur	Contributions
Damien LE DAIN	Conception du Front-end (style global, ergonomie CSS), modélisation et création de la base de données, gestion du versioning (Git)
Hamza KARROUCHI	Front-end (intégration logique JavaScript), gestion des ressources (images, animations CSS), sound design immersif, mécaniques globales du jeu côté client
Nathan LASSAUNIÈRE	Back-end (architecture Java Spring Boot, API REST, sécurité JWT, logique métier et algorithmes d'Intelligence Artificielle)

0. Table des Matières

1. Introduction

2. Architecture Logicielle Globale

2.1 Le Serveur Back-end (Spring Boot)

2.2 Modélisation de la Base de Données

3. Choix Techniques et Justifications

3.1 Front-end : Vanilla JavaScript

3.2 API RESTful et Polling Asynchrone

3.3 Gestion de l'État Serveur : RAM et ConcurrentHashMap

4. Conception Orientée Objet et Moteur de Jeu

4.1 Héritage et Polymorphisme des Animatroniques

4.2 Gestion des Collisions

5. Logique Métier et Intelligence Artificielle

5.1 Bluebear et Redbear — Comportement de base

5.2 Burncap — Comportement de "Staring"

5.3 Oneeyed — Comportement lié à l'état global

6. Fonctionnalités Front-end et Rendu

6.1 Event-Driven Audio

6.2 Mode DEV — Outils de Débogage

6.3 Easter Egg — Golden Fabron

7. Bilan, Difficultés et Perspectives

7.1 Difficultés Techniques

7.2 Évaluation des Acquis

7.3 Améliorations Futures

8. Conclusion

1. Introduction

Dans le cadre du module **R401 — Architecture Logicielle**, il nous a été demandé de concevoir et de développer un jeu vidéo web en l'espace de trois semaines. L'objectif principal était de mettre en pratique la séparation stricte des responsabilités à travers une architecture client-serveur, où un client léger interagit avec une API REST sécurisée.

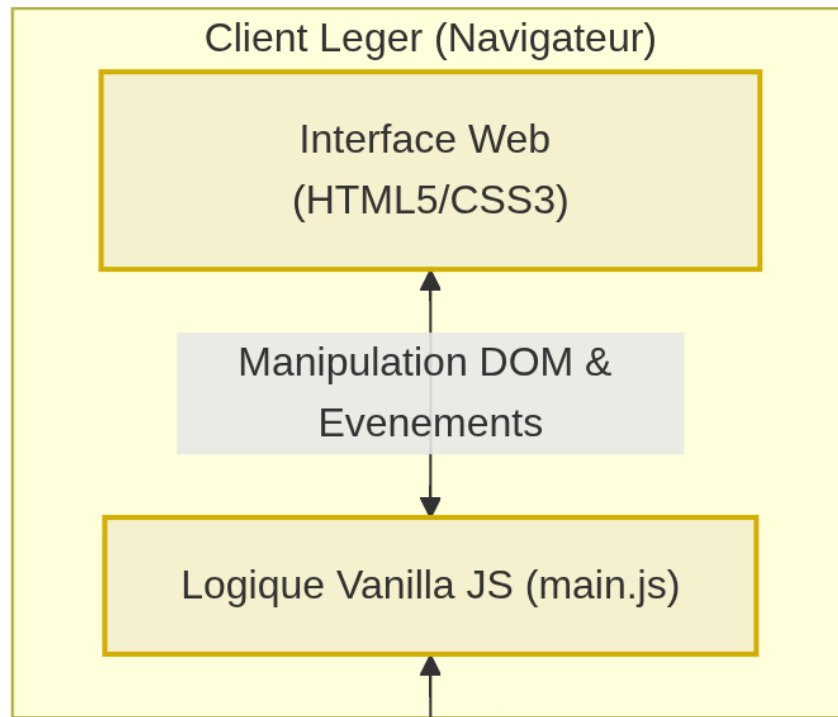
Notre choix s'est porté sur la création de « **Five Nights at Fabron** », un jeu de survie horrifique en *Point-and-Click* fortement inspiré de la franchise *Five Nights at Freddy's*. Ce genre vidéoludique se prête particulièrement bien à l'exercice d'architecture : il nécessite une gestion rigoureuse de l'état (portes, caméras, température) et des événements asynchrones (mouvements des monstres, événements sonores).

La contrainte majeure que nous nous sommes imposée est le modèle **Server-Authoritative**. Le client ne calcule aucune logique de jeu ; il se contente de capter les actions du joueur et de les transmettre au serveur. C'est le serveur, développé en Java Spring Boot, qui calcule les déplacements de l'IA, gère le temps, et renvoie l'état absolu de la partie — empêchant de fait toute triche par manipulation du code source côté navigateur.

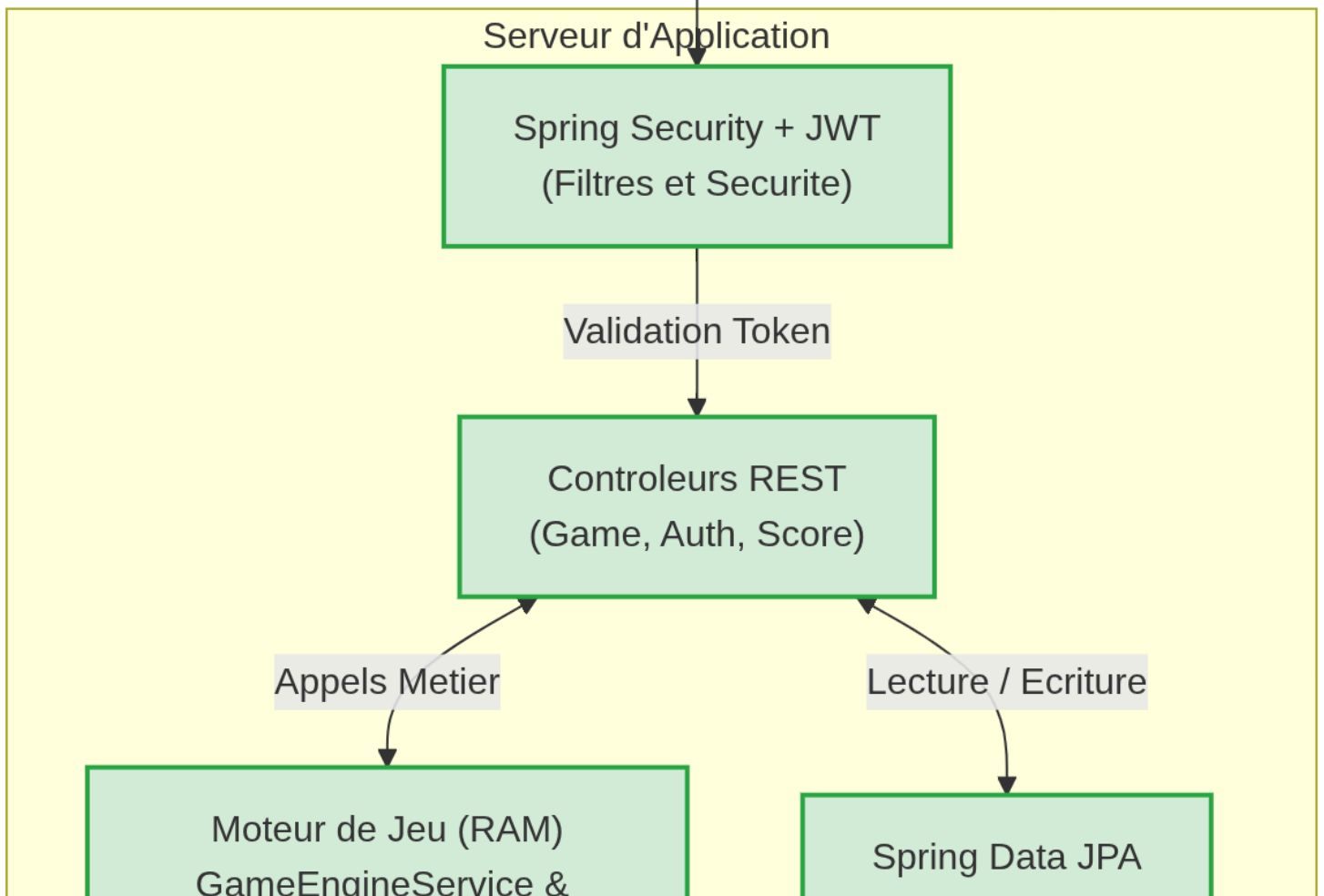
Ce rapport détaille notre architecture logicielle, justifie nos choix techniques face aux contraintes du cahier des charges, et présente en profondeur l'implémentation de nos algorithmes.

2. Architecture Logicielle Globale

Pour répondre aux exigences de robustesse et de séparation des couches, nous avons opté pour une architecture **N-Tiers** classique.



Requetes HTTP Polling
JSON



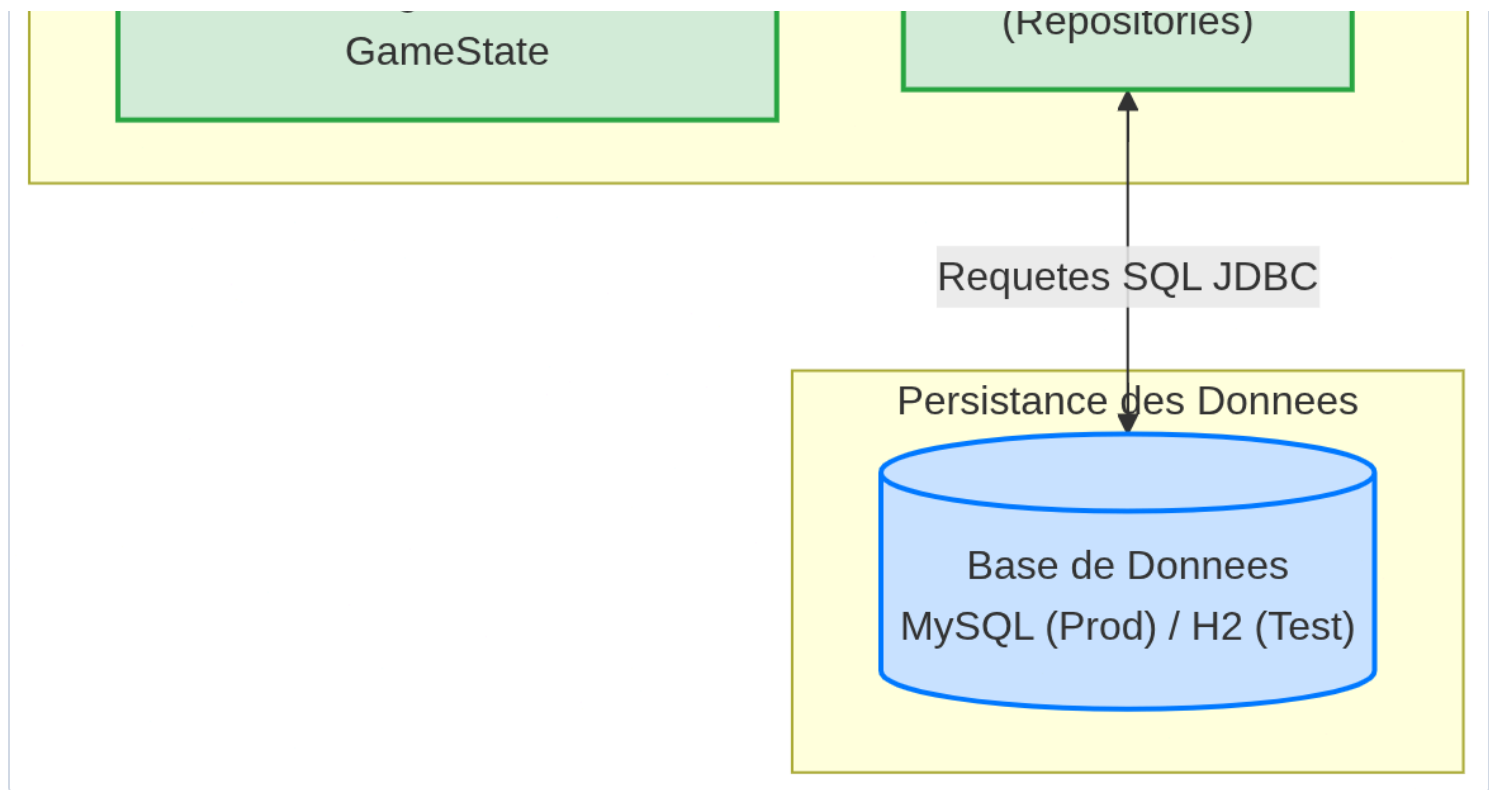


Figure 1 — Diagramme de l'architecture globale du système.

2.1. Le Serveur Back-end (Spring Boot)

Développé en **Java 17** avec le framework **Spring Boot 3.2.4**, le serveur est le cœur névralgique du jeu. Il est structuré selon les principes du *Domain-Driven Design* (DDD) avec des packages clairement isolés (`auth`, `config`, `game`, `player`, `score`). Il est découpé en quatre couches :

- **Couche de Sécurité** (`SecurityConfig`, `JwtUtil`) : Intercepte les requêtes HTTP, valide les tokens JWT et gère les droits d'accès via les filtres Spring Security.
- **Couche de Contrôle** (`Controllers`) : Expose les *endpoints* REST, reçoit les actions du client (Payload JSON) et formate les réponses.
- **Couche Service** (`GameEngineService`) : Contient la logique métier pure. L'état des parties (`GameState`) y est conservé en mémoire vive de manière concurrente.
- **Couche d'Accès aux Données** (`Repositories`) : Interfaces Spring Data JPA communiquant avec la base de données.

2.2. Modélisation de la Base de Données

La persistance repose sur un schéma relationnel géré par Hibernate (ORM). Trois entités principales ont été identifiées :

1. **Player** : Informations d'authentification (nom d'utilisateur, mot de passe haché BCrypt) et nuit maximale atteinte (`currentNight`).
2. **Score** : Associé en relation *One-to-One* avec le joueur, stocke le score global pour le *Leaderboard*.
3. **GameSession** : Trace les tentatives de survie (heure de début `startTime`, statut de la partie).

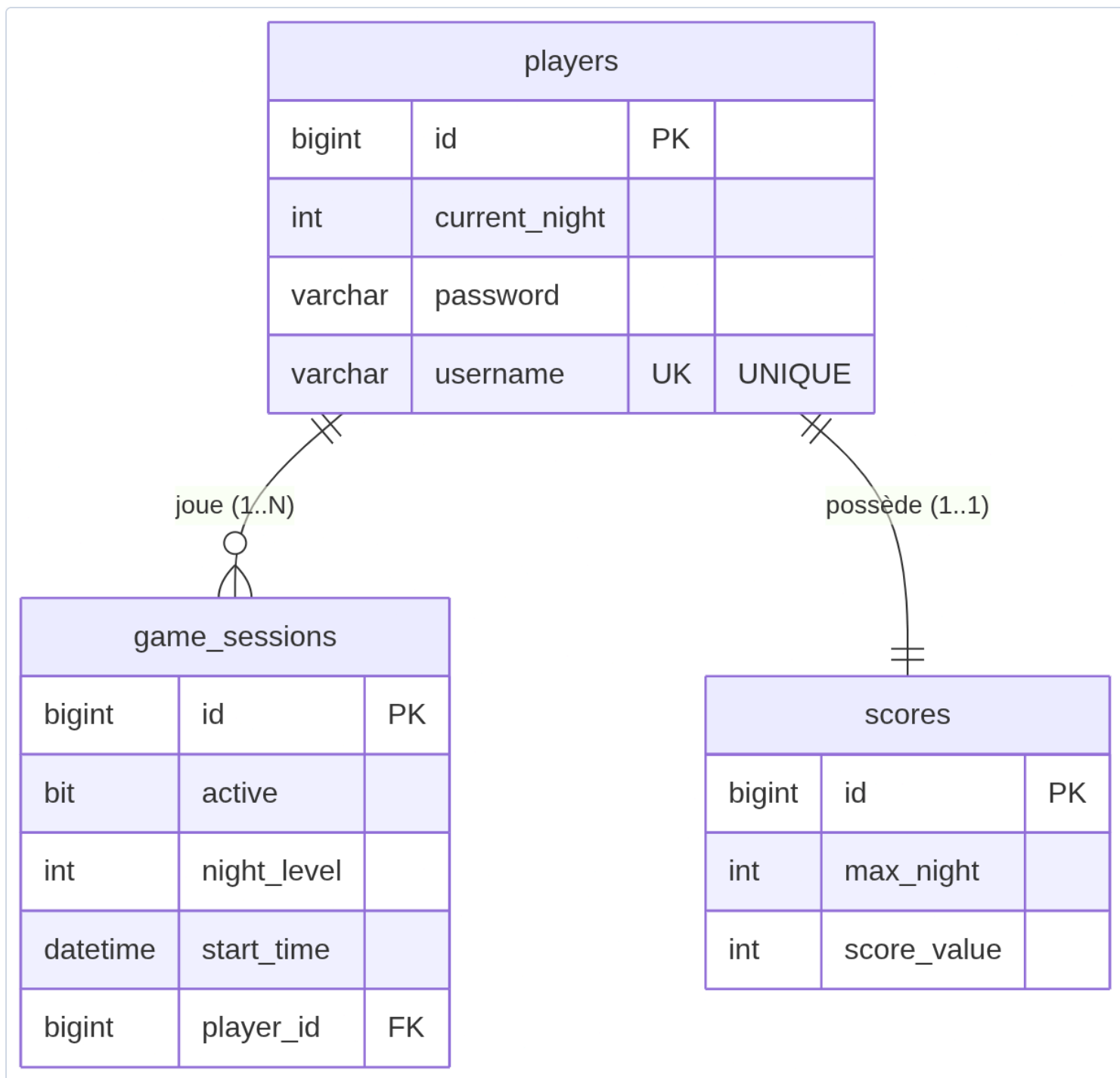


Figure 2 — Schéma entité-relation de la base de données.

3. Choix Techniques et Justifications

3.3. Front-end : Vanilla JavaScript (Sans Framework)

Conformément aux directives du cahier des charges, le front-end a été intégralement développé en **HTML5**, **CSS3** et **JavaScript natif (Vanilla ES6)**.

Justification : *Développer en Vanilla JS nous a obligés à maîtriser le DOM et l'API `fetch()` en profondeur. L'absence de Virtual DOM a permis d'optimiser les performances en manipulant directement les éléments critiques. Toute la complexité visuelle (filtres CRT, glitches) a été déportée vers le CSS pour soulager le thread JavaScript principal.*

3.4. Communication : API RESTful et Polling Asynchrone

Pour synchroniser le jeu en temps réel, nous avons mis en place une stratégie de **Polling** via l'API `fetch()`. Chaque seconde (1 000 ms), le client exécute la boucle suivante :

```
// Extrait de main.js (simplifié)
const response = await fetch('http://localhost:8080/api/game/state', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${token}` },
  body: JSON.stringify({ rightDoorClosed, leftDoorClosed, windowClosed, currentCamera })
});
const data = await response.json();
// data contient : positions, events (sons), currentHour, status...
```

Justification : *Bien que les WebSockets soient le standard pour les jeux temps réel, la nature de notre jeu (IA calculée toutes les 4 à 6 secondes) tolère un rafraîchissement par seconde. Le Polling respecte l'architecture REST demandée, est Stateless (chaque requête transporte son JWT) et simplifie la gestion de la sécurité réseau.*

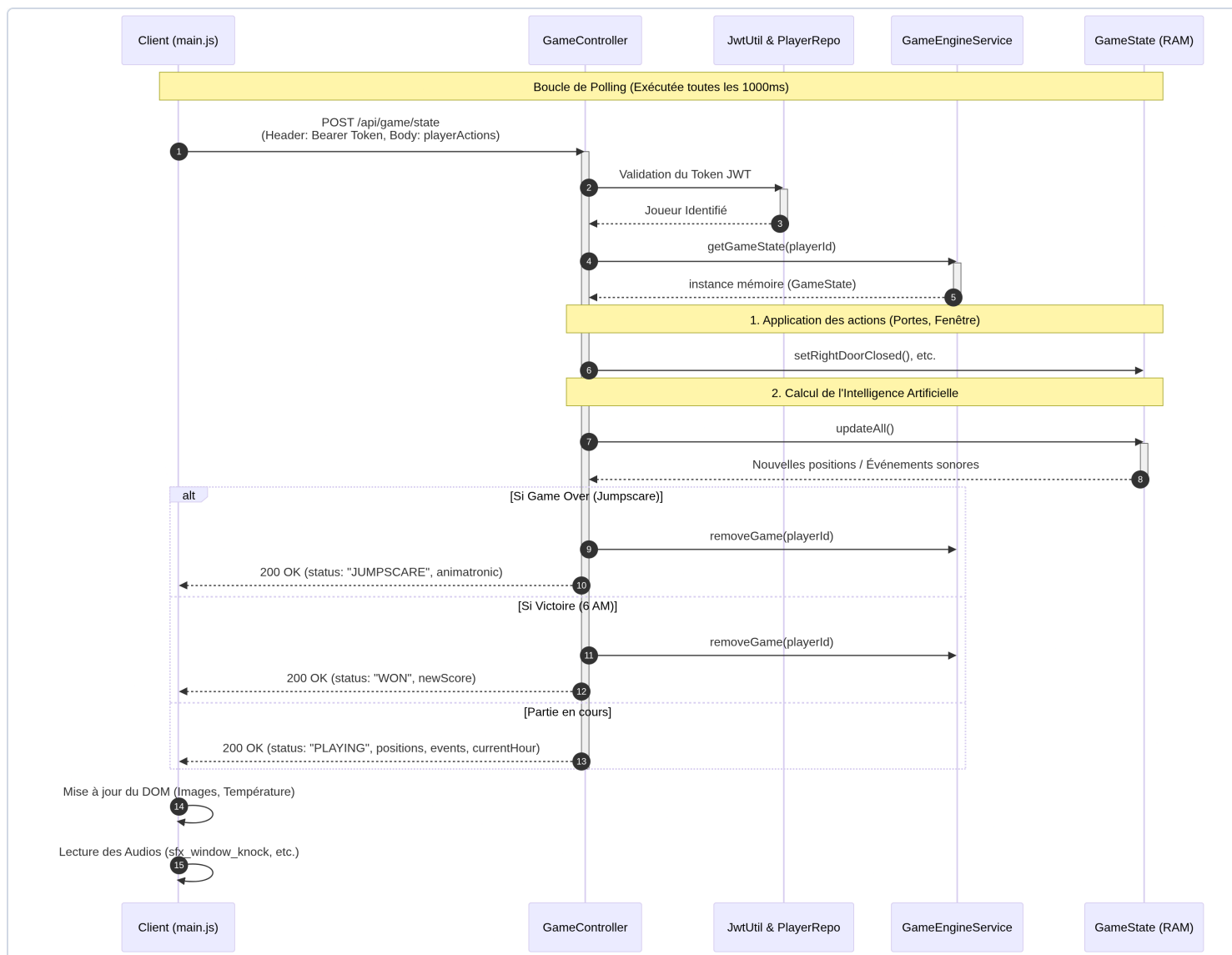


Figure 3 — Diagramme de séquence : boucle de Polling et principe Server-Authoritative.

3.5. Gestion de l'État Serveur : RAM et ConcurrentHashMap

L'état d'une partie (`GameState`) est géré par le `GameEngineService` en mémoire vive au sein d'une `ConcurrentHashMap<Long, GameState>`.

Justification : Effectuer des opérations SQL (I/O) chaque seconde pour des dizaines de joueurs aurait saturé la base de données. La RAM offre des performances en lecture de l'ordre de $O(1)$. La `ConcurrentHashMap` garantit la **Thread-Safety** dans l'environnement multi-thread de Spring Boot (Tomcat), évitant des erreurs de concurrence si deux requêtes altèrent la même partie simultanément.

4. Conception Orientée Objet et Moteur de Jeu

Pour assurer la maintenabilité et l'évolutivité du moteur en Java, nous avons appliqué les principes **SOLID**, notamment le polymorphisme et l'encapsulation.

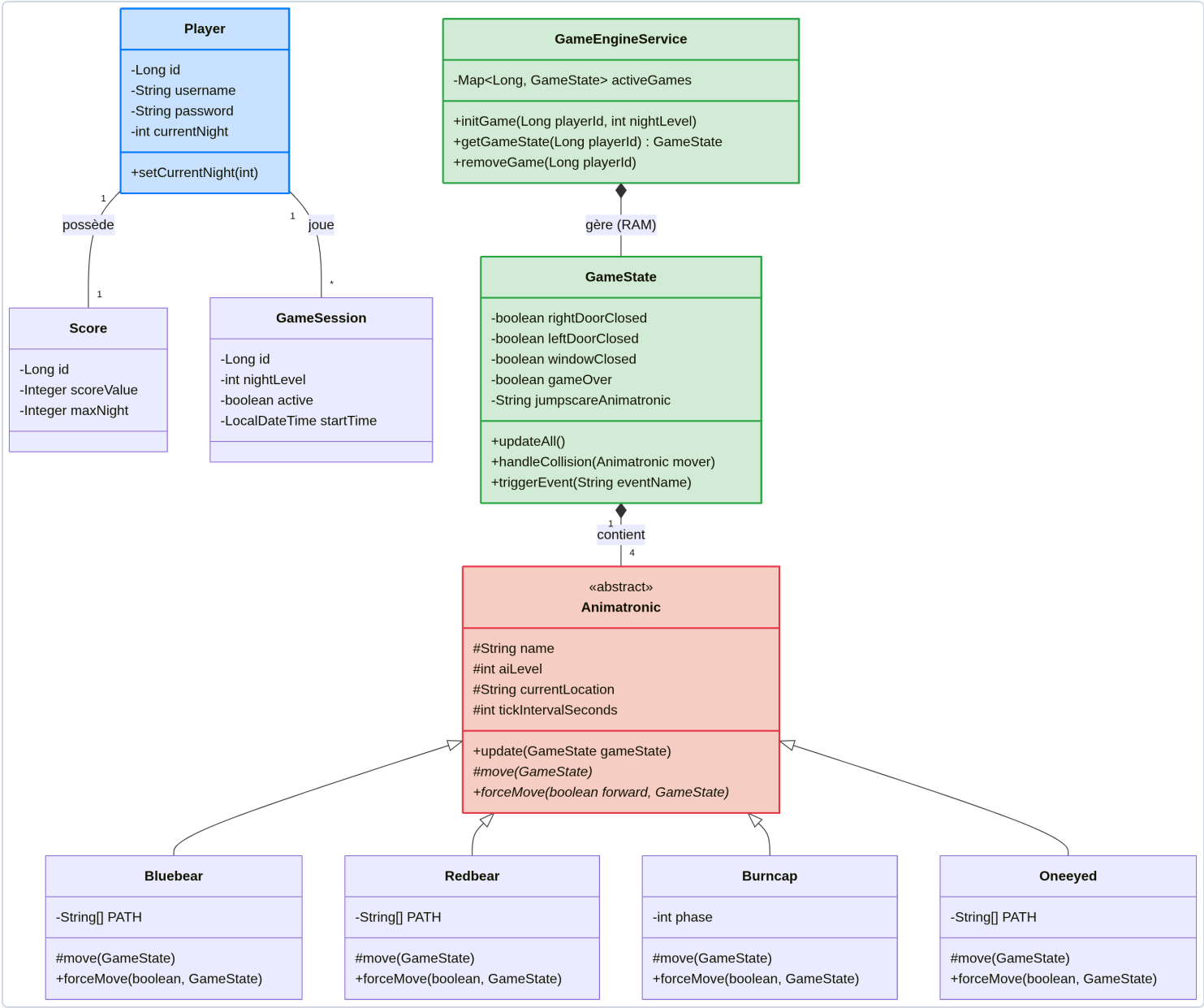


Figure 4 — Diagramme de classes : Intelligence Artificielle et Moteur de Jeu.

4.6. Héritage et Polymorphisme des Animatroniques

La logique des ennemis repose sur la classe abstraite `Animatronic`, qui définit les propriétés communes (`aiLevel`, `currentLocation`, `tickIntervalSeconds`) et la méthode principale `update(GameState)`. La gestion du temps utilise un système de probabilités inspiré d'un jet de dés D20 :

```
// Extrait de Animatronic.java
if ((currentTime - lastMoveTime) ≥ tickIntervalSeconds * 1000L) {
    this.lastMoveTime = currentTime;
    if (random.nextInt(20) + 1 ≤ aiLevel) {
        move(gameState); // Appel polymorphe
    }
}
```

Chaque monstre concret (`Bluebear`, `Redbear`, `Burncap`, `Oneeyed`) implémente obligatoirement `move(GameState)`. Ce design suit le **Principe Ouvert/Fermé** : l'ajout d'un monstre n'exige aucune modification du `GameState`.

4.7. Gestion des Collisions

Certains lieux ne peuvent accueillir qu'un seul monstre à la fois. Le `GameState` implémente `handleCollision(Animatronic mover)` : en cas de conflit spatial, la méthode `forceMove()` contraint le monstre occupant à se déplacer vers la pièce suivante, créant une réaction en chaîne organique dans le comportement de l'IA.

5. Logique Métier et Intelligence Artificielle

L'IA a été conçue pour être **asymétrique** : le joueur doit apprendre le comportement (*Pattern*) de chaque antagoniste pour espérer survivre.

5.8. Bluebear et Redbear — Comportement de base

Ces deux monstres attaquent respectivement par les portes droite et gauche, en suivant un chemin prédéfini :

```
String[] PATH = { "ext_03", "ext_02", "int_05", ... }
```

À l'index final (devant la porte), une logique temporelle s'active :

- Si la porte est fermée plus de 3 secondes → l'IA bat en retraite (`leaveDoor()`).
- Si la porte est ouverte au terme du délai d'attente (3 à 9 s, aléatoire) → `triggerJumpscare()` est appelé et la partie se termine (`setGameOver(true)`).

5.9. Burncap — Comportement de "Staring"

`Burncap` avance rapidement vers la fenêtre s'il n'est pas observé. Si la caméra active est `ext_01`, son timer est remis à zéro en permanence :

```
// Extrait de Burncap.java
boolean isBeingWatched = "ext_01".equals(gameState.getCurrentCamera());
if (isBeingWatched) {
    this.lastMoveTime = currentTime; // Remise à zéro perpétuelle du timer
}
```

5.10. Oneeyed — Comportement agressif lié à l'état global

`Oneeyed` refuse de quitter la zone de départ (`ext_03`) tant que `Bluebear` ou `Redbear` s'y trouvent. Ce mécanisme crée un système de **vagues artificielles**, empêchant le joueur d'être attaqué par tous les monstres simultanément dès le début de la nuit.

6. Fonctionnalités Front-end et Rendu

Le front-end gère un nombre important de responsabilités liées au rendu, à l'interface et à la communication réseau, sans jamais calculer de logique de jeu.

6.11. Event-Driven Audio

Le serveur ne renvoie pas uniquement des coordonnées : il émet des *événements* éphémères générés par les actions des monstres (ex. `sfx_chair_scoot` lors d'un déplacement). Le script JavaScript les intercepte et instancie des objets

`Audio` à la volée :

```
// Lecture dynamique des événements audio générés par le backend
if (data.events && data.events.length > 0) {
  data.events.forEach(eventName => {
    let audio = new Audio(`sound_effect/${eventName}.mp3`);
    audio.play();
  });
}
```

Cela permet au joueur de s'orienter dans l'espace uniquement par le son — mécanique indispensable dans les jeux d'horreur modernes.

6.12. Mode DEV — Outils de Débogage et Sécurité

Pour tester sans survivre 6 minutes réelles, des routes d'administration ont été créées dans le `GameController` (`/api/game/dev/move`, `/api/game/dev/jumpscare`). Elles ne sont accessibles que si le token JWT appartient à un développeur autorisé :

```
if (!List.of("Hamza", "Nathan", "Damien").contains(player.getUsername())) {
  return ResponseEntity.status(403).body("Accès refusé. Réservé aux développeurs.");
}
```

6.13. Easter Egg — L'Entité Golden Fabron

En effectuant certaines actions spécifiques, le joueur déclenche l'apparition de **Golden Fabron**. Le CSS gère un filtre sépia extrême et un tremblement statique (`@keyframes golden-shake`). Le JavaScript simule un plantage total de l'interface en masquant le DOM avant de rediriger le joueur vers l'écran d'accueil.

7. Bilan, Difficultés et Perspectives

7.14. Difficultés Techniques Rencontrées

Latence réseau et conflits d'état : Le modèle Server-Authoritative introduisait un décalage entre l'action du joueur (fermeture de porte, effet visuel immédiat) et sa validation serveur 1 seconde plus tard. La solution a nécessité des variables tampons côté serveur pour accorder une marge d'indulgence lors des attaques.

Chargement des assets : Charger toutes les images de caméras à la volée provoquait des clignotements blancs indésirables (*Flickering*). Nous avons résolu ce problème en préchargeant toutes les ressources lourdes en mémoire vive du navigateur au démarrage.

7.15. Évaluation des Acquis

Le projet a largement dépassé le cadre d'un simple jeu web. Il nous a permis de :

- Mettre en œuvre une architecture RESTful robuste avec Spring Boot.
- Implémenter manuellement des filtres JWT et la cryptographie BCrypt.
- Déployer une base de données relationnelle testable via JPA et H2 in-memory.
- Concevoir une IA multi-entités thread-safe en environnement concurrent.

7.16. Améliorations Futures

WebSockets : Le passage de Polling au protocole WebSocket (via STOMP/SockJS) réduirait l'empreinte réseau (*overhead* des headers HTTP) et permettrait une résolution temporelle inférieure à la milliseconde pour les Jumpscars.

Cache distribué Redis : Transférer le `GameState` de la `ConcurrentHashMap` Tomcat vers Redis rendrait l'API scalable horizontalement, permettant à plusieurs instances serveur de partager le contexte de jeu sans perte de données.

8. Conclusion

La conception de « Five Nights at Fabron » a constitué une expérience extrêmement formatrice. En appliquant rigoureusement les préceptes de l'Architecture Logicielle N-Tiers, nous avons réussi à concevoir un jeu fluide, interactif et sécurisé, où les logiques métiers complexes sont abstraites derrière des interfaces API robustes.

Le cahier des charges initial est intégralement rempli, agrémenté de fonctionnalités bonus significatives — IA asymétrique, Leaderboard, authentification JWT, design graphique avancé — qui démontrent la solidité des fondations architecturales établies en trois semaines.